

產品需求文件 (PRD)

票券通卡比價王 · Ticket & Pass Comparison · 吳思竺 Camilla Wu

第一部分：策略目標 (Strategy)

總覽 (Overview)

「票券通卡比價王」是一款由 AI 驅動的智能旅遊財務規劃工具。它旨在幫助自由行旅客，透過直覺的拖曳操作來規劃個人化行程，並利用 AI 即時獲取、分析最新的票券數據，自動計算出最經濟實惠的購買方案。本產品不僅要解決旅客在財務規劃上的核心痛點，更是我們在 AI 旅遊規劃市場中，建立技術領先地位的關鍵第一步。

問題陳述 (Problem Statement)

自由行旅客在規劃行程時，普遍面臨資訊過載與決策焦慮。面對目的地五花八門的景點門票、城市通票和交通卡，他們感到混亂且難以有效比較，經常耗費大量時間研究，卻仍不確定自己的選擇是否最划算。這種「選擇的不確定性」帶來了顯著的財務焦慮，進而影響了整體的旅遊體驗。

產品目標 (Objectives)

此 MVP 版本的產品目標具體如下：

- 驗證 AI 核心價值：**驗證 AI 核心價值：證明「視覺化行程規劃」結合「AI 自動近即時 (Near Real-time) 比價與智能總結」，能顯著提升用戶的決策效率與滿意度，成為產品的核心護城河。
- 解決核心財務痛點：**專注解決用戶在「購票前」的票券選擇困難，必須清晰地為用戶計算並標示出最划算的購買方案，直接創造「省錢」和「省心」的價值，以此建立用戶信任。
- 奠定商業模式基礎：**透過解決此高價值痛點來吸引早期用戶，為後續可能的增值服務或訂閱制商業模式，累積關鍵的用戶基礎與行為數據。

目標客群 (Personas)

1. 效率導向的規劃者 (The Efficiency-Oriented Planner) – Sarah

“我的時間應該花在享受旅行，而不是浪費在規劃旅行上。”

- 基本資料：**
- 姓名：** Sarah Chen
- 年齡：** 30歲
- 職業：** 科技業專案經理
- 背景：** Sarah 的工作節奏快、壓力大，習慣用高效率的方式解決問題。「時間就是金錢」是她的座右銘。她熱愛旅行，將其視為釋放壓力的最佳方式，但她極度厭惡行前規劃的繁瑣過程。對她來說，花好幾個晚上在不同網站間爬文、比價、拼湊行程，是一件投資報酬率極低的事情。她精通科技產品，並期待工具能像得力助手一樣，聰明地為她打理好一切。
- 目標 (Goals)：**
- 一站式完成規劃：** 希望一個地方就能完成從景點探索到票券比價的所有事情，不想在多個網站或 App 間跳轉。
- 智能推薦與決策：** 渴望工具能直接給出「最佳解」，例如最順的路線或最划算的票券，她相信 AI 的分析能力，不想陷入無謂的細節比較。
- 彈性應對變化：** 期待行程能應對突發狀況（如下雨、臨時想換點），並能即時提供合理的替代方案。

- **痛點 (Frustrations) :**
- **資訊過載與碎片化：** 網路上的旅遊資訊太多太雜，篩選和驗證資訊真偽的過程讓她感到心力交瘁。
- **制式化的無奈：** 討厭千篇一律的「懶人包」行程，但若要从零開始打造完全客製化的行程又太花時間，陷入兩難。
- **計畫趕不上變化：** 精心排好的行程常因突發狀況而打亂，但現有的規劃工具都過於僵化，無法提供動態調整的彈性。
- **使用情境 (Scenario) :**
- Sarah 計畫和朋友去日本關西自由行。她打開「票券通卡比價王」，輸入「大阪、京都」，並將她感興趣的幾個景點（如環球影城、清水寺、伏見稻荷）用滑鼠拖曳到右側的行程表中。在她拖曳的過程中，系統幾乎是即時地在下方顯示出 AI 的分析結果，並直接在一張標有「最划算方案」的卡片上，推薦她購買「關西周遊券 + 環球影城門票」的組合，同時清楚標示比全部單買省下了多少錢。她看到這個清晰的建議後，立刻就做了決定，前後不到 10 分鐘就解決了最頭痛的票務問題，然後開心地去研究要去哪家餐廳了。

2. 謹慎型的旅客 (The Cautious Traveler) – David

“旅行的安心感，來自於對花費和細節的掌握。”

- **基本資料：**
- **姓名：** David Lin
- **年齡：** 45歲
- **職業：** 會計師
- **背景：** David 是一位經驗豐富的自由行愛好者，通常負責規劃全家出遊。他對預算非常敏感，但並非吝嗇，而是追求「物有所值」。他享受研究的過程，但最困擾他的是各種票券、通卡的複雜規則，充滿了文字陷阱。他最怕的不是花錢，而是花了冤枉錢或買了不划算的產品。他對資訊的準確性有極高的要求，信任必須建立在透明、可驗證的數據之上。
- **目標 (Goals) :**
- **財務完全透明：** 清楚了解每一筆預估花費，包含交通、門票、通票的詳細費用明細，以便完全掌控預算。
- **價值最大化決策：** 獲得數據驅動的購票建議，不僅想知道哪個方案最划算，更想了解「為什麼」，以及確切能省下多少錢，幫助他做出最安心的財務決策。
- **確保資訊權威性：** 希望工具提供的所有資訊（特別是票價、包含的項目）都是準確且最新的，避免到了現場才發現規則不符的窘境。
- **痛點 (Frustrations) :**
- **規則複雜難比較：** 各種通票的規則（如使用天數、適用範圍、特定設施限制）極度複雜，難以自行比較真實的效益。
- **資訊時效性焦慮：** 網路上的分享文或官網資訊可能更新不及時，他總是擔心票價已經變動，或優惠方案已經過期。
- **害怕做出錯誤決策：** 因為資訊不足或錯誤，導致買了不划算的票券組合，進而影響整個旅程的心情和預算。
- **使用情境 (Scenario) :**
- David 正在規劃一家四口的歐洲之旅，其中一站是巴黎。他打開工具，鉅細靡遺地將所有預計要參觀的景點（羅浮宮、凡爾賽宮、聖母院等）和預計的每日地鐵搭乘次數都輸入進去。系統計算後，不僅用紫色外框標示出「最划算方案」是購買「巴黎博物館通票 + Navigo 週卡」，更用並排的卡片列出了其他幾個可行方案（例如全部單買、只買博物館通票等）的價格。David 點開了「最划算方案」的詳情，看到裡面清楚

列出了每個景點的單獨票價、通票如何覆蓋這些費用、以及交通費的估算，最後匯總出一個清晰的節省金額。看到這個詳盡的數據佐證，他感到非常放心，並將這個結果頁面截圖存檔，作為他旅行預算表的一部分。

成功指標 (Success Metrics)

- **任務完成率**：用戶成功「建立一個包含至少 3 個景點的行程，並查看 AI 比價結果」的比例。
- **AI 價值認同率**：用戶點擊查看「最划算方案」卡片詳情的比例，或在問卷中對「AI 智能總結」的滿意度分數。
- **數據準確性**：用戶回報 AI 提供的票價或通票資訊錯誤的比例，必須維持在 **0.5%** 以下。
- **用戶活躍度**：衡量用戶平均單次 session 的拖曳互動次數，以及「新增一天」的使用頻率。

• 第二部分：功能與範疇 (Features & Scope)

功能列表與優先級 (Features & Priority)

Must-have (必要功能)

- **AI 旅遊數據搜尋引擎**：用戶輸入城市，系統透過串接 Google Gemini API 與啟用 Google Search grounding，獲取景點、通票、交通票券等數據。**為平衡成本與即時性，將導入智慧快取 (Smart Caching) 機制**：當使用者搜尋時，系統優先提供時效內（例如 6-12 小時）的快取資料；若無有效快取，則觸發即時 API 呼叫以更新數據。
- **視覺化行程規劃器 (含響應式互動)**：提供雙欄佈局。**在桌面版**，用戶可將左側「景點資源庫」的門票選項，以拖曳 (Drag and Drop) 方式加入右側的多日「行程規劃區」。**為優化手機操作體驗，在行動裝置上將改為「點擊新增 (Tap-to-Add)」模式**，用戶點擊景點後選擇日期即可加入。
- **全自動比價引擎**：在用戶規劃行程的任何時刻（新增/移除景點、變更交通次數），系統都在後端自動、即時地重新計算所有可能的購買方案，並與「單獨全買」的基準方案比較。
- **AI 智能分析與結果呈現**：將比價數據傳給 AI，生成自然語言的「智能總結」，並以卡片列表形式呈現各方案。對「最划算方案」進行特殊視覺標記，並包含計算中的骨架屏動畫與完成後的懸浮按鈕引導。
- **國際化支援**：提供語言選擇器，支援繁體中文、English、日本語，確保 AI 生成的內容與介面語言一致。

Should-have (應有功能)

- **儲存行程到我的帳戶**：允許登入用戶保存已規劃的行程，方便後續查看與修改。

Could-have (可有功能)

- **分享行程連結**：產生一個唯讀的行程與比價結果分享連結，方便用戶與旅伴溝通。
- **替代景點建議**：當用戶加入一個景點時，AI 可建議附近或性質相似的其他熱門景點。

Won't-have (排除功能)

- **沉浸式預算守門員**：不包含詳細的預算輸入、超支提醒或實時消費追蹤功能。MVP 階段聚焦於「購票前」的比價決策，而非「旅途中」的記帳。
- **社群驅動消費共筆**：不引入用戶上傳消費數據或攻略分享的功能。為確保數據權威性，初期數據來源完全由 AI 引擎控制。
- **線下地圖或導航功能**：不提供 App 內的地圖導航，專注於解決財務規劃問題。

使用者故事 (User Stories)

- **(AI 搜尋)** 作為一個行程規劃者，我想要輸入一個城市名就能立即看到所有相關的景點和票券，以便我能快速開始規劃，而不用在多個網站間跳轉。

- (視覺化規劃) 作為效率導向的 Sarah，我想要用拖曳的方式將景點安排到每日行程中，以便我能直觀地組織行程，並輕鬆地調整順序。
- (自動比價) 作為謹慎型的 David，我想要每當我調整行程時，系統都能自動重新計算並顯示最省錢的方案，以便我能隨時確保我的選擇是當下最經濟的。
- (AI 總結) 作為一個不想看複雜數據的旅客，我想要直接閱讀一段話，告訴我哪個方案最好以及為什麼，以便我能快速做出決定，並對這個決定感到放心。

使用者流程與介面 (User Flow & UI)

1. **搜尋與載入**：用戶在首頁輸入城市名，系統顯示動態載入效果，並透過 AI 引擎即時獲取數據，將景點、通票等資訊填充至左側的「景點資源庫」。
2. **規劃行程**：在桌面版，用戶從左側資源庫中，將感興趣的景點門票拖曳到右側的「日期卡片」中。在手機版，用戶點擊資源庫中的景點，在彈出選項中選擇要加入的日期。用戶可以新增天數、設定每日交通次數。
3. **即時比價**：每當行程變動，後端比價引擎會自動觸發，計算所有方案。此時介面會顯示骨架屏動畫，表示正在計算中。
4. **查看結果**：計算完成後，出現「查看比價結果」懸浮按鈕。點擊後，頁面滾動至結果區塊。頂部是 AI 生成的自然語言總結，下方是並列的方案卡片。最划算的方案會以紫色外框和頂部標籤醒目標示，清晰展示總花費與節省金額。(詳細 Wireframe 與 UI 設計稿將另行提供。)

• 第三部分：技術與執行 (Implementation)

實作與整合方案 (Implementation and Integration Plan)

鑑於產品對使用者體驗（即時拖曳、流暢互動）與數據準確性（即時比價）的高度要求，我們將採納 **Code-based MVP (程式開發最小可行產品)** 的策略。

我們的核心開發哲學將從原先的「Vibe Coding」模式，全面升級為「**脈絡工程 (Context Engineering)**」。這意味著我們不單是追求用自然語言快速產出程式碼，而是將開發重心放在**設計、管理並優化傳遞給 AI 的資訊供應鏈**上。我們的目標是打造一個可預測、可維護且高效的 AI 系統，其中 AI 不僅是個黑盒子，而是一個被精準引導的核心組件。

核心理念：以脈絡工程 (Context Engineering) 取代 Vibe Coding

「Vibe Coding」擅長解決單點問題，而「脈絡工程」則著眼於整個系統的穩定與可靠。具體作法包含：

1. **結構化提示詞鏈 (Structured Prompt Chaining)**: 我們會將複雜的任務拆解成一系列更小、更明確的 AI 請求。每個請求的輸出 (Output Context) 將成為下一個請求的輸入 (Input Context)，形成一條清晰的「脈絡鏈」。
2. **數據脈絡化 (Data Contextualization)**: 傳遞給 AI 的不是零散的文字，而是結構化、帶有明確意義的數據 (如 JSON)，讓 AI 能在最沒有歧義的狀況下進行分析與生成。
3. **系統角色脈絡 (System Role Context)**: 在 Prompt 中明確告知 AI 它的角色 (例如「你是一個專業的旅遊財務顧問」)、任務目標以及期望的輸出格式，確保其產出符合系統需求。

分階段執行計畫 (Phased Execution Plan)

我們將開發過程分為三個核心階段：

第一階段：脈絡設計與架構 (Context Design & Architecture)

- **目標**：在撰寫任何程式碼前，先完成 AI 系統的藍圖設計。
- **任務**：

1. **定義核心脈絡鏈 (Core Context Chain):** 繪製出從「使用者輸入城市」到「AI 生成最終總結」的完整資訊流圖 (Context Flow Diagram)。明確標出每一個環節的輸入與輸出脈絡。
2. **撰寫提示詞腳本 (Prompt Scripts):** 根據脈絡鏈，撰寫兩個核心的 Prompt 腳本：
 - `Prompt_Search_v1`: 用於「AI 旅遊數據搜尋引擎」，定義如何將使用者輸入的城市名轉換為對 Gemini API 的精準搜尋指令。
 - `Prompt_Summary_v1`: 用於「AI 智能分析」，定義如何將後端比價引擎計算出的結構化數據 (JSON)，轉換為給用戶的自然語言總結。
3. **定義數據結構 (Data Schemas):** 明確定義 `行程規劃資料`、`比價結果資料` 等在系統中流動的 JSON 物件的格式，這將是 AI 理解任務的基礎。
 - **產出物:** Context Flow 圖、Prompt 腳本文件、數據結構規格書。

第二階段：原型驗證與迭代 (Prototyping & Iteration)

- **目標:** 快速驗證第一階段設計的 Prompt 腳本是否能產生預期結果，並進行優化。
- **任務:**
 1. **單元化測試 Prompt:** 使用 Gemini API 的開發者工具 (如 Google AI Studio) 或簡單的腳本，直接測試 `Prompt_Search_v1` 和 `Prompt_Summary_v1` 的效果。
 2. **數據格式驗證:** 準備多組模擬的 `比價結果資料` JSON，餵給 `Prompt_Summary_v1`，檢查 AI 是否能穩定地理解數據並生成高品質的總結。
 3. **RAG 效果評估:** 針對 `Prompt_Search_v1`，重點評估 Gemini API 的 Search grounding 功能是否能準確抓到最新的票價與通票資訊。
- **產出物:** 經過驗證且優化後的 Prompt 腳本 v2 版本、關鍵 API 呼叫的範例程式碼。

第三階段：系統實作與整合 (System Implementation & Integration)

- **目標:** 將驗證過的脈絡鏈與 Prompt，透過程式碼整合進完整的應用程式中。
- **任務:**
 1. **搭建資訊管道 (Build the Pipelines):** 開發前後端程式，建立起在第一階段設計好的「資訊管道」，確保數據能按照 Context Flow 順暢流動。
 2. **實作比價引擎:** 開發 `comparisonService.ts`，使其能精準接收前端傳來的行程資料，並產出標準格式的結構化比價結果 JSON。
 3. **整合 AI 呼叫:** 將第二階段驗證過的 Prompt 腳本與 API 呼叫邏輯，實作到後端程式碼中，串接起整個自動化流程。
 4. **開發前端介面:** 打造視覺化行程規劃器，確保使用者互動能即時觸發後端的脈絡鏈。
- **產出物:** 功能完整的 MVP 產品。

技術棧 (Tech Stack)

- **開發工具 (Cursor):** 其對整個 Codebase 的上下文理解能力至關重要。在開發時，AI 不僅是生成新程式碼，更能理解現有程式碼的脈絡，幫助我們將新的功能模組無縫整合進既有的「資訊管道」中。
- **前端 (Next.js):** 作為使用者脈絡的起點。它負責捕捉使用者的意圖 (拖曳景點、調整天數)，並將其轉化為結構化的數據脈絡，傳遞給後端。
- **後端 (Next.js API Routes + Vercel Postgres):** 這是脈絡處理與調度的中樞。它負責接收前端脈絡、執行核心業務邏輯 (比價引擎)、調度對 AI 核心的請求，並管理智慧快取 (Smart Caching) 以優化脈絡的時效。

性與成本。

- **AI 核心 (Google Gemini Pro API):** 我們的**核心脈絡處理引擎**。
- **搭配 Search grounding:** 扮演**外部脈絡獲取器 (External Context Retriever)** 的角色，從公開網路獲取即時數據。
- **核心模型:** 扮演**脈絡理解與生成器 (Context Understanding & Generator)** 的角色，將結構化的數據脈絡轉化為人類易於理解的自然語言。

核心運作流程 (Context Chain in Action)

此流程更清晰地展示了「提示詞鏈」的運作方式：

1. 脈絡起點 (User Input):

- **Context_In_1:** 使用者在前端輸入城市名，例如 `{ "city": "東京" }`。

2. 提示詞鏈 #1 (數據搜尋):

- 前端將 `Context_In_1` 送至後端。
- 後端觸發 **Prompt_Search_v2**: 指示 Gemini API (啟用 Search grounding) 搜尋與該城市相關的景點、通票和交通票券的最新資訊。
- **Context_Out_1:** Gemini API 回傳結構化的原始數據 JSON，並存入快取。

3. 脈絡演進 (User Interaction):

- 前端將 `Context_Out_1` 渲染為景點資源庫。
- 使用者進行拖曳操作，前端狀態更新。
- **Context_In_2:** 前端將當前的行程規劃狀態打包成 JSON，例如：`{ "days": 2, "transport_rides": [3, 4], "attractions": ["景點A", "景點B", "景點C"] }`。

4. 核心邏輯處理 (Non-AI):

- 前端將 `Context_In_2` 即時發送到後端的「全自動比價引擎」。
- 比價引擎執行其內部演算法，計算出所有購買方案。
- **Context_Out_2:** 比價引擎產出一個純粹的、結構化的比價結果 JSON，例如：`{ "best_option": "方案X", "saved_amount": 3500, "details": [...] }`。

5. 提示詞鏈 #2 (智能總結):

- 後端將 `Context_In_2` (使用者行程) 和 `Context_Out_2` (比價結果) 合併，作為一個完整的數據脈絡。
- 後端觸發 **Prompt_Summary_v2**: 指示 Gemini API 根據提供的行程與比價數據，生成一段自然語言的「智能總結」。
- **Context_Out_3 (最終結果):** Gemini API 回傳總結文字。後端將此文字與 `Context_Out_2` 一起打包回傳給前端渲染。

非功能性需求 (Non-functional Requirements) – **性能 (Performance):** 整體端到端 (從操作到看見結果) 的響應時間目標在 **2–4 秒內**。比價計算應在 1 秒內完成，AI 生成時間則透過流式傳輸優化體感。 – **數據準確性與即時性 (Accuracy & Timeliness):** 數據來源直接由 Gemini API 搭配 Google Search grounding 提供。透過智慧快取機制 (例如 6–12 小時的 TTL)，在營運成本與數據新鮮度之間取得平衡，確保用戶獲取的絕大多數資訊為半日內更新，滿足規劃需求。

- **穩定性 (Reliability):** 系統穩定性依賴於部署平台 (如 Vercel/Netlify) 的服務狀態與我們自身的程式碼品質，擁有比多個 No-code 工具串接更高的可控性。

待解決問題與決策紀錄 (Open Issues & Decision Log) – 待解決問題: – (已部分解決) Gemini API 在高頻次呼叫下的成本評估與用量監控策略。→ 已透過引入智慧快取機制進行初步應對。 – (新問題) 需進行數據分析，以釐清智慧快取的最佳時效 (TTL)，在用戶體驗與成本間找到最佳平衡點。 – 決策紀錄:

- 決策: 由 No-code MVP 轉向 Code-based MVP

理由: 為了實現產品的核心功能 (即時拖曳、即時比價)，並確保產品的長期擴充性與優質使用者體驗。原 No-code 方案在功能完整度、性能與數據即時性上已無法滿足產品最終願景。

- 決策: 採用 Cursor 作為主要開發工具

理由: 最大化 AI 輔助開發的效率，降低程式開發的入門門檻，並實現最流暢的「Vibe Coding」開發流程。

- 決策: 引入智慧快取 (Smart Caching) 機制

理由: 為了在 MVP 階段有效控制高頻次 API 呼叫的營運成本，同時又能為使用者提供足夠新鮮、準確的數據 (例如半日內)，避免完全即時帶來的高昂費用，也避免每日批次更新造成的數據嚴重延遲。

技術實作與開發環境 (成本優化版)

為確保 Context Engineering 專案的開發成本控制在最低，同時保持設計與實作效率，採用以下工具策略 (含現有 GPT Plus 訂閱)：

工具與階段配置

階段 主要任務 主力工具 輔助工具 花錢策略

*1. 構思與設計階段 – 梳理需求與專案範圍 – 撰寫 PRD / Prompt 腳本 – 建立 Context Flow 框架

GPT-4.1 (Plus) → 穩定長上下文 + 文件產出 Claude 免費版 (大篇推理備援)、Gemini Pro 免費 3 個月不買 Claude Pro，靠 GPT-4.1 長對話能力完成設計

2. 原型與驗證階段 – 把 Prompt 實作成程式碼 – 測試 API 串接、簡單 RAG – 快速修正邏輯錯誤

Cursor 免費版 (內建 IDE 快速驗證) GPT-4.1 (Plus) 輔助 Debug & 優化 不升級 Cursor Pro，免費額度用完就切本地 VSCode / Colab

3. 上線與優化階段 – 大規模測試 – 整合多檔案與長上下文 – 自動化 workflow

GPT-4.1 (Plus) 持續主力

Cursor 免費版 (代碼驗證)、Claude 免費版 (特殊推理)

保持現有 GPT Plus 訂閱即可，其他維持免費

額度節省策略

1. 小測試走免費工具，大推理走 GPT Plus。
2. 免費額度用完前切到另一個免費工具 (Claude / Gemini)。
3. 文件產出與代碼運行分流處理 (GPT-4.1 整理文檔、Cursor 寫程式)。

✂ 全年額外成本：

- GPT Plus (已訂閱，無需額外支出)
- 其他工具皆用免費版 → 額外成本 \$0
- 若後期要極速疊代才考慮短期升級 Cursor Pro (按月 \$20，用一兩個月即可)